

fid-inc-smoke____fidelity-binomial-f-k4-inc

An AgentMPI run analysis

Abstract

This is a smoke test: 47 events, 0.062s, a `function` executor on all eight ranks, run ten minutes before the broker campaign it was checking. Its job was to confirm that the new `-incompressible` payload generator and the new value-level retention metrics plumb through the harness end to end, and it did: the row it produced is the first in the repository to carry `n_intact`, `n_label_only`, `n_dropped`, `value_retention` and `id_retention`, at 96, 0, 0, 1.0 and 1.0. The structural counts are exact — `fold_depth`: $3 = \lceil \log_2 8 \rceil$, 3 rounds, 7 messages $= p - 1$, 7 applications, model check passing — and the retention figure of 1.000 must not be read as a fidelity result. Nothing here could have lost anything: the surrogate kernel ignores the token budget entirely, so its outputs grow from 386 tokens at depth 1 to 3,033 at the root against a nominal 450, and there is no budget-bound accumulator for incompressible content to overflow. What little the kernel did discard is almost comic: the four depth-1 merges all have prompts of exactly 1,695 characters, the kernel seeds its generator from `seed + len(prompt)`, and replaying its arithmetic shows all four making the same draw and dropping candidate index 0 — which is not a fact but the prompt’s own instruction sentence, “`such as [F-3-2]. Keep every identifier ...`”. The smoke test therefore established that the plumbing works and that a surrogate executor cannot exercise erasure at all. The measurement it cleared the way for is `inc-mb-fidelity-inc`, which added `max_tokens`: 450 to the contract, ran on real agents, and returned 0.3854 with ranks 4 through 7 erased whole.

1 What this run is

property	value
run	fid-inc-smoke__fidelity-binomial-f-k4-inc
experiment	–
campaign label	–
declared world size	8
ranks appearing in the log	8
executors	<code>function</code> $\times$ 8
events	47
wall time	0.06 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	0.02×	total busy time over wall time
parallel efficiency	0.2%	achieved parallelism per rank
idle fraction	99.8%	rank-seconds paid for and unused
peak ranks busy	1	of 8 in the world
serial fraction of active time	100.0%	time with exactly one rank busy
load imbalance	8.00×	slowest rank’s busy time over the mean
coordination share	22.2%	rank-seconds blocked in collectives, of those available
coordination in flight	91.5%	wall clock with any rank inside a collective
rank reattachments	0	fresh agent processes that took over a rank role
agent calls	7	invocations that reached a model
agent latency p50 / p95	0.00 / 0.00 s	per invocation
messages	7	7 eager, 0 rendezvous
tokens sent	2,579	0 deferred under rendezvous
tokens in / out	5,782 / 6,649	prompt and completion
cost	\$0.1171	0.0176 \$/kTok out

3 What the analysis notices

- **[note]** Occupancy is not measurable for this run: its executors (function) emit no broker claim events, so busy time, achieved parallelism, and the idle fraction are artefacts of the instrumentation rather than properties of the run.
- **[note]** Too few distinct output sizes to fit β , so the reported latency parameters are medians rather than a regression.
- **[note]** Load imbalance is 8.00×: the slowest rank was busy far longer than the mean.
- **[ok]** All 1 checkable collective(s) sent exactly the number of messages their closed-form cost expression predicts.
- **[ok]** All 1 checkable collective(s) also matched the predicted round count.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	0.00	1.7	1	3	0	3	0	0.0	finalized
1	0.00	0.0	0	0	1	0	178	0.0	finalized
2	0.00	0.0	0	1	1	1	406	0.0	finalized
3	0.00	0.0	0	0	1	0	178	0.0	finalized
4	0.00	0.0	0	2	1	2	1,055	0.0	finalized
5	0.00	0.0	0	0	1	0	178	0.0	finalized
6	0.00	0.0	0	1	1	1	406	0.0	finalized
7	0.00	0.0	0	0	1	0	178	0.0	finalized

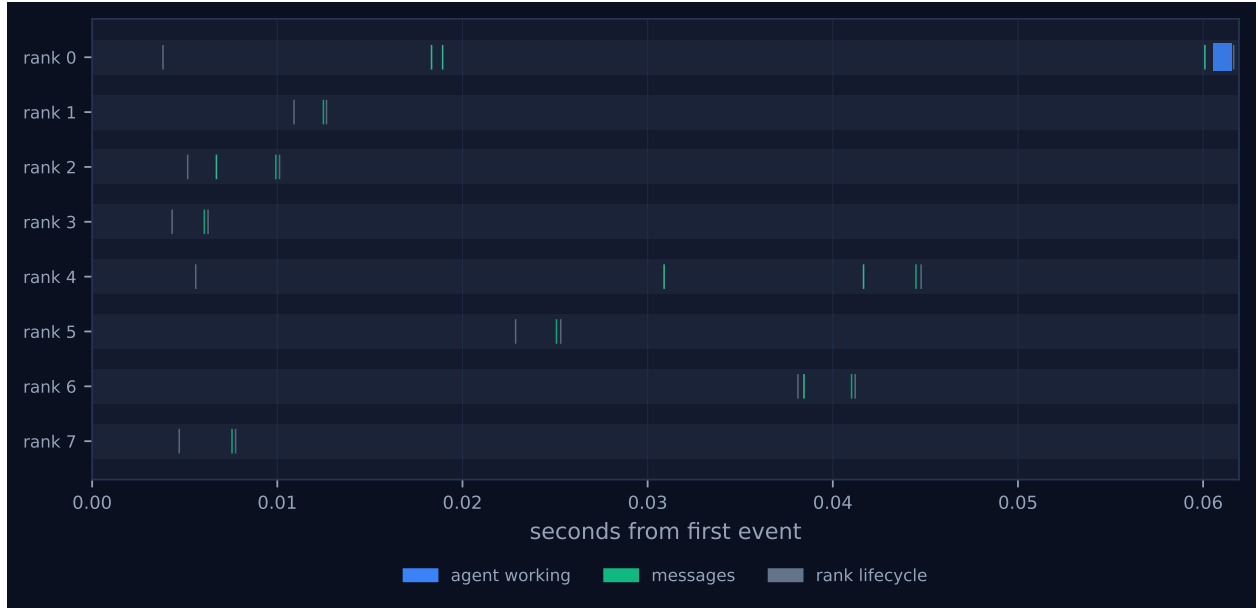


Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer’s palette so the same shapes read the same way in both.

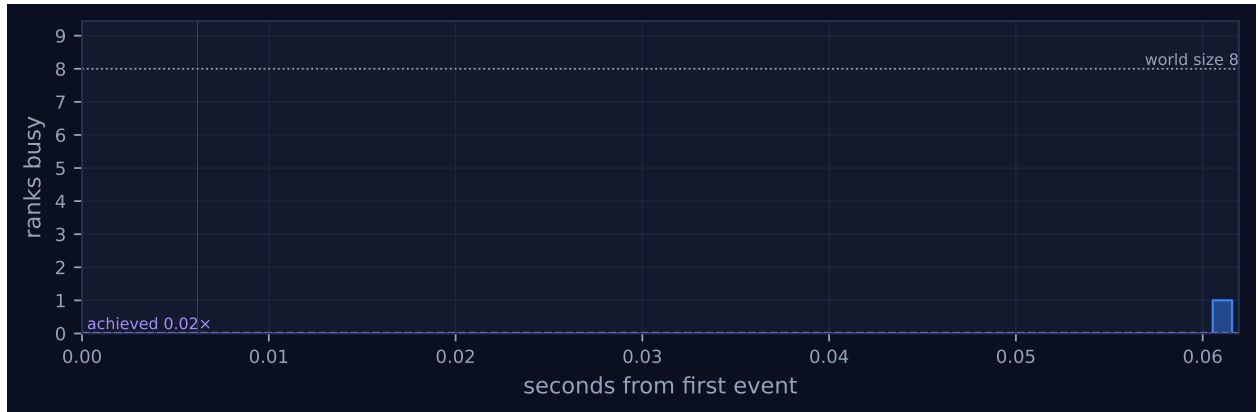


Figure 2: Ranks simultaneously busy over time. The dotted line is the declared world size and the dashed line the achieved parallelism; the gap between them is what the run paid for and did not use. Faint vertical lines mark collective invocations.

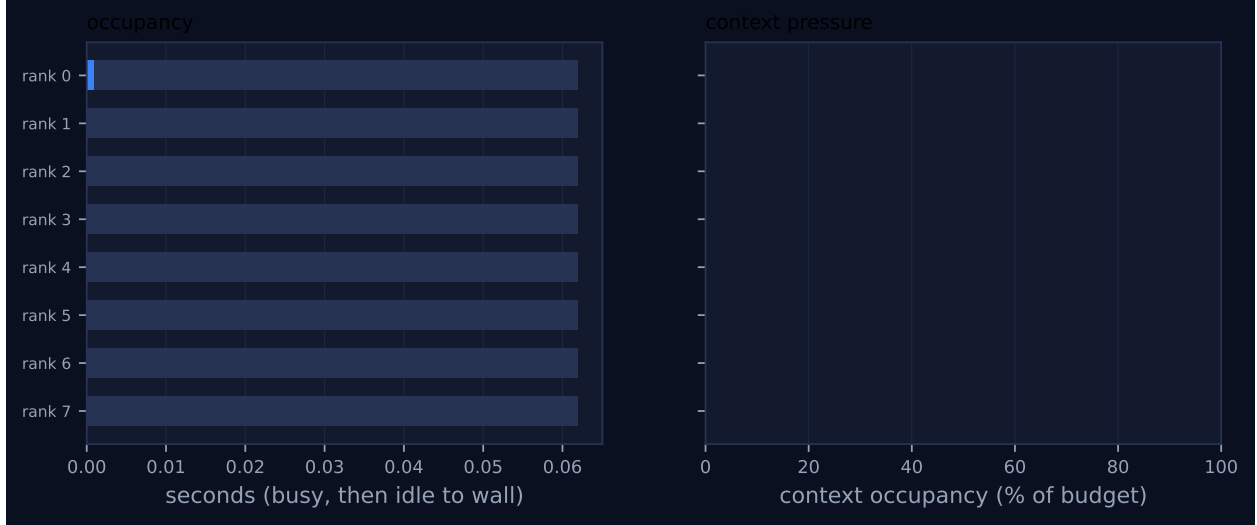


Figure 3: Per-rank occupancy and context pressure. Left: busy time, with the remainder to wall time in grey. Right: context occupancy against budget, amber above half and red above 80%, where admission pressure starts to reject artefacts.

Per-rank behaviour. Occupancy is busy time over the rank’s own lifetime, so a rank that joined late is not penalised for the time before it existed.

5 Collectives, against the cost model

op	algorithm	label	p	seen	rounds	pred.	msgs	pred.	tokens	wall (s)	skew (s)	model
reduce	binomial	–	8	8	3	3	7	7	2,440	0.06	0.06	✓

Messages are the count the fabric logged, attributed to each invocation through its internal tag, rather than the count the collective reported — the two are independently produced, and preferring the log is what makes this a check rather than a restatement. A dash in the model column means the comparison is not meaningful: either no closed form exists for that algorithm, or the invocation never completed.

6 Reading the timeline

Figure 1 is a binomial tree at millisecond resolution and contains no surprises: four ranks act at depth 1 (0, 2, 4, 6), two at depth 2 (0, 4), one at depth 3, and the whole run is 0.062s with every application an instant. The per-rank call counts descend 3, 0, 1, 0, 2, 0, 1, 0, which is the `stride` loop’s signature, and the seven messages of the communication matrix, Figure 4, carry 178 tokens on each leaf edge and 406 and 1,055 on the two accumulated edges into the root, 2,579 in total. The $8.00\times$ imbalance flagged in the findings is the concentration a tree produces at $p = 8$ and is expected; the occupancy row of the dashboard should be ignored, because these executors emit no broker claim events and the generated findings say so directly.

The one thing the widths do show is that the operator is not respecting its budget. The four depth-1 calls take 446-token prompts and return 386; the two depth-2 calls take 900 and return 1,036; the

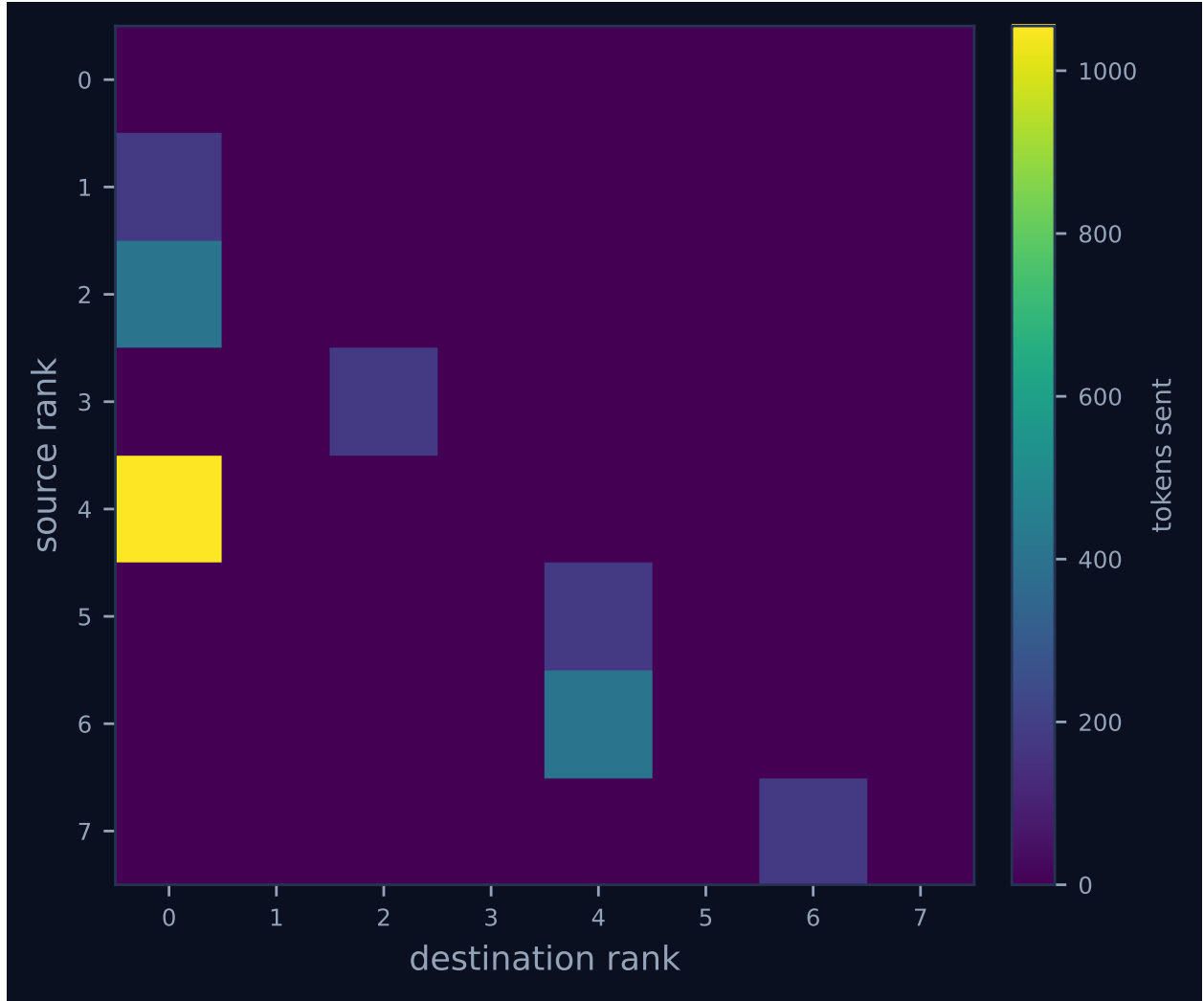


Figure 4: Communication matrix: tokens sent from each rank to each rank. The algorithm’s shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

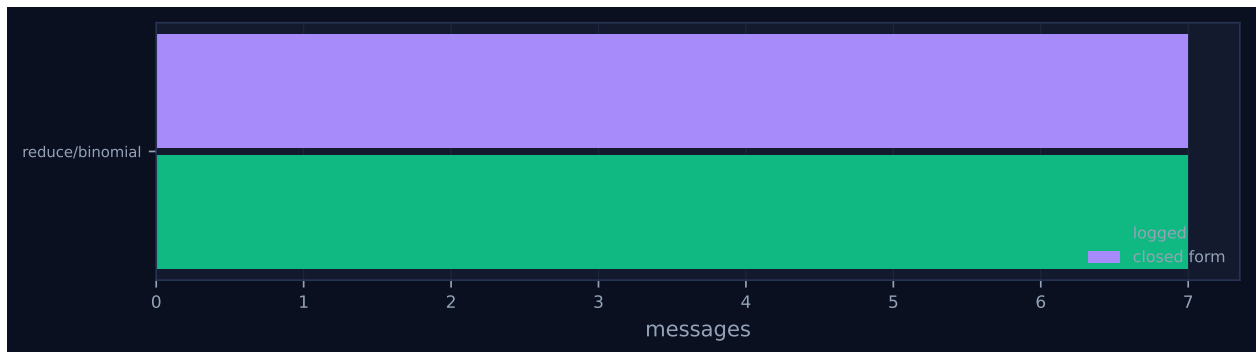


Figure 5: Logged message count against the closed-form prediction, per invocation. A red diamond marks a disagreement between the implementation and its own cost model.

root takes 2,198 and returns 3,033. Output grows faster than input because the kernel copies both of its inputs verbatim into its result — the stored accumulator for the $4 \rightarrow 0$ message is JSON nested inside escaped JSON inside escaped JSON — so the run emits 6,649 output tokens for a fold over 96 twelve-token facts, against a stated ceiling of 450 per application.

7 Interpretation

The structure is by construction and is what a smoke test should verify: depth 3 matching $\lceil \log_2 8 \rceil$, seven messages against a predicted seven, and the finding that all checkable collectives agree with their closed form is the check passing. What the run adds to the campaign is a negative result about its own instrument, and it is worth stating because it is the reason the campaign was re-run on a broker.

Retention 1.000 here has two independent causes and neither is fidelity. The first is that the surrogate has no notion of a budget: `_surrogate_merge` takes a drop rate and a seed, never the token ceiling, and the ceiling was in any case advisory in this run — at commit 90ca469 the harness passed the module-level `MERGE_CONTRACT`, which has no `max_tokens` field, so the 450 tokens existed only as a sentence in the prompt and as a `meta` hint. Whole-rank erasure in this experiment is what happens when an accumulator that stands for many ranks fills a binding budget; with no binding budget and no budget-aware operator, the mechanism cannot fire, and `-incompressible` — which removes the operator’s ability to re-encode its way out of pressure — has nothing to act on. The second cause is the surrogate’s actual loss model, which is not the one its docstring describes. It builds candidates from `prompt.splitlines()` and keeps lines matching `FACT_ID`, but each input is interpolated as `json.dumps(...)` and contains no newlines, so a binary merge prompt offers exactly three candidates: the prompt’s instruction line containing the example identifier `[F-3-2]`, and the two inputs entire. The unit of loss is a whole input, and the draw is a deterministic function of prompt character length. Rebuilding the four depth-1 prompts from `make_fact_report` at the run’s seed gives 1,695 characters for every one of them, and the replay keeps candidates 1 and 2 and drops candidate 0 in all four cases — the instruction sentence. The stored blob for the $2 \rightarrow 0$ message confirms it: two findings, 24 identifiers, ranks 2 and 3 complete, and no `[F-3-2]`. So the only thing this run lost was harness boilerplate, four times, deterministically.

Read against my other fourteen runs, this cell is therefore a plumbing datum and a caution rather than evidence. The comparison it makes possible is the important one: `inc-mb-fidelity-inc__fidelity-binomial-f-h` is the same algorithm at the same p , the same twelve incompressible items per rank and the same nominal 450 tokens, run ten minutes later, differing in exactly two respects — a real broker executor, and `max_tokens: 450` in the contract with a semantics clause instructing the operator to omit whole items rather than re-encode. It retains 37 of 96: ranks 0, 1 and 2 complete, one item of rank 3, and ranks 4 through 7 gone entirely. That is the measurement the experiment was built for, and this smoke test is the step that confirmed the new metric block would record it.

8 Threats to this reading

The executor is a surrogate, so nothing here is agent behaviour, and the cost figure of \$0.1171 is a modelled price for tokens produced by string concatenation. The 0.062s wall time is SQLite writes and event logging.

The replay is a reconstruction rather than a logged fact: a `function` executor writes no rows to

`agent_calls`, so this run’s fabric contains no prompts and no results, only the seven message payloads under `blobs/`. What is directly observed is that those payloads contain nested escaped JSON, that their `findings` arrays have length 2 or 3 rather than 12 or 24, and that the $2 \rightarrow 0$ payload carries 24 identifiers with the phantom absent. The claim that all four depth-1 prompts were 1,695 characters and made the same draw comes from rebuilding them with the harness’s own generator, which reproduces the observed output-token counts (386 for all four) and the observed blob contents.

Two defects I would report rather than fix. The prompt’s example identifier `[F-3-2]` is a live identifier that `fact_retention` will score if the operator echoes it; in this run it does no harm, but in `fid-synth_fidelity-binomial-f-k4` it is the sole reason rank 3 is credited with 0.25 retention after its real report was destroyed. And the line-based item extraction makes the surrogate’s loss granularity a whole subtree rather than a fact, which means the $(1 - r)^{\text{depth}}$ curve its docstring promises is not what any of these runs exhibit.